

Dafny – another industrial application

Dafny is a verification-aware, full-grown programming language with support for common programming concepts such as inductive datatypes, lambda expressions, and classes. It can be compiled to C#, Java, Javascript, Python, and Go. It is used at Amazon Web Services to analyse, e.g.

- resource policies,
- access management,
- storage infrastructures,
- find bugs,
- guide testing.
- reInforce: proven correct Authorisation engine for AWS, Called 1B/sec, c.f.,
<https://www.youtube.com/watch?v=oshxAJGrwMU>
- improved the throughput of RSA signatures on Amazon's Graviton2 chip by 33% to 94%,
<https://www.amazon.science/blog/formal-verification-makes-rsa-faster-and-faster>

More at

<https://www.amazon.science/research-areas/automated-reasoning>.

Slide 1

Proving Properties of Operational Semantics

We will want to prove assertions like those below:

1. $\forall E \in Expr, n \in \mathbb{N}, s, s' \in State.$

$$[\langle E, s \rangle \Downarrow_e \langle n, s' \rangle \iff \langle E, s \rangle \rightarrow_e^* \langle n, s' \rangle].$$
2. $\forall B \in Bool, s, s' \in State, bv \in \mathbb{B}.$

$$[\langle B, s \rangle \Downarrow_b \langle bv, s' \rangle \iff \langle B, s \rangle \rightarrow_b^* \langle bv, s' \rangle].$$
3. $\forall C \in Comm, s, s' \in State.$

$$[\langle C, s \rangle \Downarrow_c \langle s' \rangle \iff \langle C, s \rangle \rightarrow_c^* \langle skip, s' \rangle].$$

These assertions are about many different entities:

states $s, s' \in State$,

terms $E \in Expr$, and values $bv \in \mathbb{B}, n \in \mathbb{N}$

relations $\rightarrow_e \subseteq (Expr \times Store) \times (Expr \times Store)$,

$\rightarrow_e^* \subseteq (Expr \times Store) \times (Expr \times Store), \dots$ and

$\Downarrow_e \subseteq (Expr \times Store) \times \mathbb{N}, \Downarrow_e \subseteq (Expr \times Store) \times \mathbb{N} \dots$

Because $E, n, \rightarrow_e, \rightarrow_b, \rightarrow_c, \rightarrow_e^*, \rightarrow_b^*, \rightarrow_c^*$, and $\Downarrow_e, \Downarrow_b, \Downarrow_c$ are inductively defined, they give rise to inductive principles.

Slide 2

Proving Properties of Operational Semantics -2

We will want to prove assertions like those below:

1. $\forall E \in Expr, n \in \mathbb{N}, s, s' \in State.$

$$[\langle E, s \rangle \Downarrow_e \langle n, s' \rangle \iff \langle E, s \rangle \rightarrow_e^* \langle n, s' \rangle].$$
2. $\forall B \in Bool, s, s' \in State, bv \in \mathbb{B}.$

$$[\langle B, s \rangle \Downarrow_b \langle bv, s' \rangle \iff \langle B, s \rangle \rightarrow_b^* \langle bv, s' \rangle].$$
3. $\forall C \in Comm, s, s' \in State.$

$$[\langle C, s \rangle \Downarrow_c \langle s' \rangle \iff \langle C, s \rangle \rightarrow_c^* \langle skip, s' \rangle].$$

For example, we could try to prove 1. from above, by induction over the term E , or by induction over the value n , or by induction over the relation $\Downarrow_e$, or by induction over the relation $\rightarrow_e^*$.

Choosing whether to prove by induction, and if so, the entity on which to apply the induction is the first, crucial step. In the Reasoning Course we have seen how to do that in the context of programs. In the Models course we will revisit the same question but in the context of entities defined through BNF, or through inference rules.

Slide 3

Proving Properties of Operational Semantics - 3

In the remainder of this unit, we shall discuss

- Proofs by induction over terms,
eg proofs over E , or n .
- Proofs by induction over relations,
e.g. proofs over $\rightarrow_e$, or $\rightarrow_b$, or $\rightarrow_c$, or $\rightarrow_e^*$, or $\rightarrow_b^*$, or $\rightarrow_c^*$,
or $\Downarrow_e$, or $\Downarrow_b$, or $\Downarrow_c$.
- Proofs that do not use induction.

Slide 4

Induction – recap

Last year, in Course 141: ‘Reasoning about Programs’, we learnt how to use structural induction to prove properties about recursively defined terms, in particular Haskell functions. We use a lot of inductive techniques in this course, both to give definitions and to prove facts about our semantics. So, it’s worth taking a little while to refresh our memories about this technique.

When designing an algorithm to solve a problem, we want to know that the result produced by the algorithm is correct, regardless of the input. For example, the quicksort algorithm takes a list of numbers and puts them into ascending order. In this example, we know that the algorithm operates on a *list of numbers*, but we do not know how long that list is or exactly what numbers it contains. Similarly, one may raise questions about depth-first search of a tree: how do we know it always visits all the nodes in a tree if we do not know the exact size and shape of the tree?

In examples such as these, there are two important facts about the input data which allows us to reason about arbitrary inputs:

- the input is *structured*: for example, a non-empty list has a first element and a ‘tail’, which is the rest of the list, and the binary tree has a root node and two subtrees;
- the input is finite.

In this situation, the technique of *induction* provides a principle by which we may formally reason about arbitrary lists, trees, and so on.

What is induction for? – recap from last year

Induction allows us to work with *infinite collections* of objects (things!) which are

- *structured* in some well-defined way;
- *finite* but *arbitrarily large and complex*.

Induction exploits the finite, structured nature of these objects to overcome their arbitrary complexity.

Slide 5

These kinds of structured, finite objects arise in many areas of computer science. Data structures such as lists and trees are common, but in fact programs themselves can be seen as structured finite objects. This means that induction can be used to prove facts about *all programs in a certain language*.

We can use induction... – recap from last year

... to reason about things like

- *natural numbers*: each one is finite, but there are infinitely many of them;
- *data structures* such as lists, trees and so on;
- *programs* in a programming language: again, you can write arbitrarily large programs, but they are always finite;
- *derivations* o like $E \Downarrow m$, or $E \rightarrow_e n$, or $E \rightarrow_e^* q$. These derivations are finite applications of the rules of $\Downarrow$, resp. $\rightarrow_e$, or $\rightarrow^*$.

Slide 6

Induction over terms

Slide 7

Induction for Nat – recap from last year**Reminder: Induction for Nat**

The set Nat can be defined by the following grammar:

$$N \in Nat ::= zero \mid succ(N).$$

This gives rise to the following inductive principle, for any $P \subseteq \mathbb{N}$:

$$P(\text{zero})$$

$$\wedge$$

$$\forall n \in Nat. [P(n) \Rightarrow P(\text{succ}(n))]$$

$$\Rightarrow$$

$$\forall n \in Nat. [P(n)]$$

Notice the similarity with the Haskell definition:

```
data Nat = Zero | Succ Nat
```

Slide 8

Architecture of a Proof by induction on $N \in \text{Nat}$

Proving that

$$\forall n \in \text{Nat}. P(n)$$

by induction on the structure of n :

Base Case:

To Show: $P(\text{zero})$

... proof of base case ...

Inductive Case:

Take any $m \in \text{Nat}$, arbitrary

Inductive Hypothesis: $P(m)$

To Show: $P(\text{succ}(m))$

proof of inductive step ... using the inductive hypothesis.

Slide 9

Inductive Definitions over Nat

We define

$$\text{plus} : \text{Nat} \times \text{Nat} \rightarrow \text{Nat},$$

a function that takes two natural numbers and returns their sum,

inductively over the function's first argument:

$$\text{plus}(\text{zero}, N) = N$$

$$\text{plus}(\text{succ}(M), N) = \text{succ}(\text{plus}(M, N))$$

Slide 10

Example proof by induction over *Nat* – architecture

We prove that

$$\forall N \in \text{Nat}. [\text{plus}(N, \text{zero}) = N]$$

by induction on *N*.

Base Case: To Show: $\text{plus}(\text{zero}, \text{zero}) = \text{zero}$.

... proof of base case ...

Inductive Case: Take $K \in \text{Nat}$, arbitrary

Inductive hypothesis: $\text{plus}(K, \text{zero}) = K$.

To Show: $\text{plus}(\text{succ}(K), \text{zero}) = \text{succ}(K)$.

... proof of inductive case ...

Slide 11

Example proof by induction over *Nat* – 1

Proof

By induction on the structure of *N*:

Base Case: To Show $\text{plus}(\text{zero}, \text{zero}) = \text{zero}$

Proof of base case

$$\text{plus}(\text{zero}, \text{zero}) = \text{zero} \quad (\text{by def. of plus})$$

Inductive Case: Take $K \in \text{Nat}$, arbitrary.

Inductive hypothesis: $\text{plus}(K, \text{zero}) = K$

Show: $\text{plus}(\text{succ}(K), \text{zero}) = \text{succ}(K)$

Proof of inductive case:

$$\begin{aligned} \text{plus}(\text{succ}(K), \text{zero}) &= \text{succ}(\text{plus}(K, \text{zero})) \quad (\text{def. of plus}) \\ &= \text{succ}(K) \quad (\text{by IH}) \end{aligned}$$

Slide 12

Induction over Binary Trees

Binary trees are a commonly used data structure. Roughly, a binary tree is either a single *leaf node*, or a *Node node* which has two *subtrees*.

Induction on the Syntax for Binary Trees

Binary trees defined by the following grammar:

$$\text{bTree} \in \text{Tree} ::= \text{Leaf} \mid \text{Node}(\text{bTree}, \text{bTree})$$

Inductive principle for binary trees:

...

$\wedge$

$\forall \dots [\dots]$

$\implies$

$\forall \text{bt} \in \text{Tree}. P(\text{bt})$

Slide 13

Induction on the Syntax for Binary Trees

Binary trees defined by the following grammar:

$$bTree \in Tree ::= Leaf \mid Node(bTree, bTree)$$

Inductive principle for binary trees:

$$P(Leaf)$$

$$\wedge$$

$$\forall bt_1, bt_2 \in Tree. [P(bt_1) \wedge P(bt_2) \Rightarrow P(Node(bt_1, bt_2))]$$

$$\Rightarrow$$

$$\forall bt \in Tree. P(bt)$$

Notice the similarity with the Haskell definition:

```
data Tree a = Leaf a | Node (Tree a) a (Tree a)
```

Slide 14

Inductive Definitions over Trees

Define functions

$$\text{leaves}, \text{nodes} : Tree \rightarrow \mathbb{N}$$

which return the number of leaves, resp. nodes in a tree:

$$\text{leaves}(Leaf) = 1$$

$$\text{leaves}(Node(bt_1, bt_2)) = \text{leaves}(bt_1) + \text{leaves}(bt_2).$$

$$\text{nodes}(Leaf) = 0$$

$$\text{nodes}(Node(bt_1, bt_2)) = \text{nodes}(bt_1) + \text{nodes}(bt_2) + 1.$$

Slide 15

Exercise

Slide 16

Prove by induction on the structure of trees that:

$$\forall bt \in \text{Tree}. \text{leaves}(bt) = \text{nodes}(bt) + 1.$$

The proof is by induction on the structure of bt .

Base Case: To Show: $\text{leaves}(\text{Leaf}) = \text{nodes}(\text{Leaf}) + 1$.

Proof of Base Case:

$$\begin{aligned} \text{leaves}(\text{Leaf}) &= 1 && \text{(by def. leaves)} \\ &= 0 + 1 && \text{by arithmetic} \\ &= \text{nodes}(\text{Leaf}) + 1 && \text{(by def. nodes)} \end{aligned}$$

Inductive Case: Take $bt_1, bt_2 \in \text{bTree}$, arbitrary.

I.H.: $\text{leaves}(bt_1) = \text{nodes}(bt_1) + 1$

$\wedge$

$\text{leaves}(bt_2) = \text{nodes}(bt_2) + 1$

To show: $\text{leaves}(\text{Node}(bt_1, bt_2)) = \text{nodes}(\text{Node}(bt_1, bt_2)) + 1$

Proof of Inductive Case:

$$\begin{aligned} \text{leaves}(\text{Node}(bt_1, bt_2)) &= \\ &= \text{leaves}(bt_1) + \text{leaves}(bt_2) && \text{(by def. leaves)} \\ &= \text{nodes}(bt_1) + 1 + \text{nodes}(bt_2) + 1 && \text{(by I.H.)} \\ &= (\text{nodes}(bt_1) + \text{nodes}(bt_2) + 1) + 1 && \text{(by arithmetic)} \\ &= \text{nodes}(\text{Node}(bt_1, bt_2)) + 1 && \text{(by def. nodes)} \end{aligned}$$

Induction over Simple Expressions

The syntax of our illustrative language *SimpleExp* also gives a collection of structured, finite, but arbitrarily large objects over which structural induction may be used.

Recall that n ranges over the natural numbers $0, 1, 2, \dots$. This means that, in this language, there are in fact an infinite number of indecomposable expressions; contrast this with the cases above, where `zero` is the only indecomposable natural number, and `Leaf` is the only indecomposable binary tree. Also, note that we can build new expressions from old in two ways, by using $+_s$ and $\times_s$.

Syntax of Simple Expressions

Simple expressions are defined as below

$$E \in \text{SimpleExp} ::= n \mid E +_s E \mid E \times_s E$$

where $n \in \mathbb{N}$ ranges over the natural numbers $0, 1, 2, \dots$

Principle of Induction for Simple Expressions

$$E \in SimpleExp ::= n \mid E +_s E \mid E \times_s E$$

Inductive Principle

$$\forall n \in \mathbb{N}. P(n)$$

$\wedge$

$$\forall E1, E2 \in SimpleExp. [P(E1) \wedge P(E2) \Rightarrow P(E1 +_s E2)]$$

$\wedge$

$$\forall E1, E2 \in SimpleExp. [P(E1) \wedge P(E2) \Rightarrow P(E1 \times_s E2)]$$

$\Rightarrow$

$$\forall E \in SimpleExp. [P(E)]$$

Slide 18

Some Properties of $\Downarrow$ for SimpleExp

Determinacy: no evaluations to more than one answer:

$$\forall E \in SimpleExp. \forall n_1, n_2 \in \mathbb{N}. \\ [E \Downarrow n_1 \wedge E \Downarrow n_2 \Rightarrow n_1 = n_2] .$$

Totality: all simple expressions evaluate to at least one answer:

$$\forall E \in SimpleExp. \exists n \in \mathbb{N}. [E \Downarrow n] .$$

Both properties can be proven by induction on the structure of expressions.

Slide 19

We give the proof of determinacy here. Totality is left as an exercise (see answers to exercise sheet on induction).

Theorem (Determinacy of $\Downarrow$)

$$\forall E \in \text{SimpleExp}. \forall n_1, n_2 \in \mathbb{N}. [E \Downarrow n_1 \wedge E \Downarrow n_2 \Rightarrow n_1 = n_2].$$

Proof. by induction on the structure of the expression E .

Base Case:

To Show: $\forall n \in \mathbb{N}. \forall n_1, n_2 \in \mathbb{N}. [n \Downarrow n_1 \wedge n \Downarrow n_2 \Rightarrow n_1 = n_2]$

Proof of Base Case

Take $n \in \mathbb{N}$, arbitrary.

Take $n_1, n_2 \in \mathbb{N}$, arbitrary.

- (1) Assume that $n \Downarrow n_1 \wedge n \Downarrow n_2$.
- (2) The derivation $n \Downarrow n_1$ was done through application of rule (B-NUM)
The derivation $n \Downarrow n_2$ was done through application of rule (B-NUM)
(by the $\Downarrow$ operational semantics of *SimpleExp*).
- (3) $n = n_1 \wedge n = n_2$ (by (2) and the rule (B-NUM))
- (4) $n_1 = n_2$ (by (3) and properties of $=$).

First Inductive Step:

Take $E_1, E_2 \in \text{SimpleExp}$, arbitrary.

Inductive Hypothesis:

$$\forall n_a, n_b \in \mathbb{N}. [E_1 \Downarrow n_a \wedge E_1 \Downarrow n_b \Rightarrow n_a = n_b]$$

$\wedge$

$$\forall n_c, n_d \in \mathbb{N}. [E_2 \Downarrow n_c \wedge E_2 \Downarrow n_d \Rightarrow n_c = n_d]$$

To Show: $\forall n_1, n_2. [E_1 +_s E_2 \Downarrow n_1 \wedge E_1 +_s E_2 \Downarrow n_2 \Rightarrow n_1 = n_2]$

Proof of First Inductive Step:

Take $n_1, n_2 \in \mathbb{N}$, arbitrary.

- (1) Assume that $E_1 +_s E_2 \Downarrow n_1 \wedge E_1 +_s E_2 \Downarrow n_2$.
- (2) The derivation $E_1 +_s E_2 \Downarrow n_1$ was done through application of rule (B-ADD)
The derivation $E_1 +_s E_2 \Downarrow n_2$ was done through application of rule (B-ADD)
(by the $\Downarrow$ operational semantics of *SimpleExp*).
- (3) There exist $k_a, k_b, k_c, k_d \in \mathbb{N}$, such that:
 $E_1 \Downarrow k_a \wedge E_2 \Downarrow k_b \wedge n_1 = k_a +_s k_b$
 $\wedge$
 $E_1 \Downarrow k_c \wedge E_2 \Downarrow k_d \wedge n_2 = k_c +_s k_d$
(by (2) and the rule (B-ADD))
- (4) $k_a = k_c \wedge k_b = k_d$ (by application of IH on (3))
- (5) $n_1 = n_2$ (by (3) and (4))

Second Inductive Step:

Take $E_1, E_2 \in \text{SimpleExp}$, arbitrary

Inductive Hypothesis Take $E_1, E_2 \in \text{SimpleExp}$, arbitrary.

Inductive Hypothesis:

$$\forall n_a, n_b \in \mathbb{N}. [E_1 \Downarrow n_a \wedge E_1 \Downarrow n_b \Rightarrow n_a = n_b]$$

$\wedge$

$$\forall n_c, n_d \in \mathbb{N}. [E_2 \Downarrow n_c \wedge E_2 \Downarrow n_d \implies n_c = n_d]$$

To Show: $\forall n_1, n_2. [E_1 \times_s E_2 \Downarrow n_1 \wedge E_1 \times_s E_2 \Downarrow n_2 \implies n_1 = n_2]$

Proof of Second Inductive Step:

This case follows the same pattern as the previous case, so we omit the details. $\square$

We could have done this proof by induction on the structure of the *derivation* $\Downarrow$, and the proof would have been a bit more succinct. In the next lectures we will see how to prove by induction on the structure of the derivation.

Challenge

How do we prove that $\rightarrow^*$ is strongly normalizing?

Slide 21

Summary: Proofs by Induction over Terms

- State over which entity is the induction – this requires thought.
- Base Case (s): State what is to be shown
- Prove the Base Case (s)
 - State what is taken arbitrary – if any
 - Justify each step
- Inductive Step (s)
 - State what is taken arbitrary (at least one thing is).
 - State the Inductive Hypothesis.
 - State what is to be shown.
- Prove the Inductive Step (s).
 - state what is taken arbitrary – if any.
 - Justify each step.
- Not all lemmas require a proof by induction.

Slide 22

Induction over Relations

1 Induction over the structure of the derivation

Induction over boolean terms

Recall...

$$B \in Bool ::= b \mid B \& B \mid \neg B \quad b \in \mathbb{B} ::= \text{true} \mid \text{false}$$

With small-steps semantics, $\rightarrow_o \subseteq Bool \times Bool$

Slide 23

$$\text{Bs1} \frac{}{\text{false} \& B_2 \rightarrow_o \text{false}}$$

$$\text{Bs2} \frac{}{\text{true} \& B_2 \rightarrow_o B_2}$$

$$\text{Bs3} \frac{B_1 \rightarrow_o B'_1}{B_1 \& B_2 \rightarrow_o B'_1 \& B_2}$$

$$\text{Bs4} \frac{B \rightarrow_o B'}{\neg B \rightarrow_o \neg B'}$$

$$\text{Bs5} \frac{}{\neg \text{true} \rightarrow_o \text{false}}$$

$$\text{Bs6} \frac{}{\neg \text{false} \rightarrow_o \text{true}}$$

Induction over boolean terms - 2

And with big-step semantics, $\Downarrow_o \subseteq \text{Bool} \times \mathbb{B}$

$$\text{BL1} \frac{B_1 \Downarrow_o \text{false}}{B_1 \& B_2 \Downarrow_o \text{false}}$$

$$\text{BL2} \frac{B_1 \Downarrow_o \text{true} \quad B_2 \Downarrow_o b}{B_1 \& B_2 \Downarrow_o b}$$

$$\text{BL3} \frac{B \Downarrow_o \text{true}}{\neg B \Downarrow_o \text{false}}$$

$$\text{BL4} \frac{B \Downarrow_o \text{false}}{\neg B \Downarrow_o \text{true}}$$

$$\text{BL5} \frac{}{b \Downarrow_o b}$$

We want to prove:

$$\forall B \in \text{Bool}. \forall b \in \mathbb{B}. [B \Downarrow_o b \implies B \rightarrow_o^* b]$$

Slide 24

Motivation: Inductive Principle for *Bool*

$P(\text{true})$

$\wedge$

$P(\text{false})$

$\wedge$

$\forall B_1, B_2 \in \text{Bool}. [P(B_1) \wedge P(B_2) \implies P(B_1 \& B_2)]$

$\wedge$

$\forall B \in \text{Bool}. [P(B) \implies P(\neg B)]$

$\implies$

$\forall B \in \text{Bool}. [P(B)]$

Slide 25

1.1 Proving that for the Booleans $\Downarrow_o$ is sound wrt $\rightarrow_o^*$

We want to prove that

$$(*) \quad \forall B \in \text{Bool}. \forall b \in \mathbb{B}. [B \Downarrow_o b \implies B \rightarrow_o^* b]$$

For the proof we will need the following lemmas:

Lemma 1. $\forall B, B' \in \text{Bool}. [B \rightarrow_o^* B' \implies \neg B \rightarrow_o^* \neg B']$

Lemma 2. $\forall B_1, B'_1, B_2 \in \text{Bool}. [B_1 \rightarrow_o^* B'_1 \implies B_1 \& B_2 \rightarrow_o^* B'_1 \& B_2]$

We postpone the proof of these lemmas for later, and proceed with the proof of (*).

Proof. of (*) is by induction over the structure of B .

Base Case -1:

To Show: $\forall b \in \mathbb{B}. [\text{true} \Downarrow_o b \implies \text{true} \rightarrow_o^* b]$

Proof of Base Case -1

Take $b \in \mathbb{B}$, arbitrary.

- (1) Assume that $\text{true} \Downarrow_o b$.
- (2) $b = \text{true}$ by inversion of def. of $\Downarrow_o$
- (3) $\text{true} \rightarrow_o^* \text{true}$, since $\rightarrow_o^*$ is reflexive
- (4) $\text{true} \rightarrow_o^* b$, by (2) and (3)

Base Case -2:

To Show: $\forall b \in \mathbb{B}. [\text{false} \Downarrow_o b \implies \text{false} \rightarrow_o^* b]$

Proof of Base Case -2

analogous to Base Case-1.

Inductive Case - 1:

Take $B_1, B_2 \in \text{Bool}$, arbitrary.

Inductive Hypothesis:

$$\forall b_1 \in \mathbb{B}. [B_1 \Downarrow_o b_1 \implies B_1 \rightarrow_o^* b_1]$$

$\wedge$

$$\forall b_2 \in \mathbb{B}. [B_2 \Downarrow_o b_2 \implies B_2 \rightarrow_o^* b_2]$$

To Show: $\forall b \in \mathbb{B}. [B_1 \& B_2 \Downarrow_o b \implies B_1 \& B_2 \rightarrow_o^* b]$

Proof of Inductive Case-1:

Take $b \in \mathbb{B}$, arbitrary.

- (1) Assume that $B_1 \& B_2 \Downarrow_o b$.
- (2) We obtained (1) through application of BL1 or BL2, (by inversion on $\Downarrow_o$).

1st Case

- (3) We applied BL1.
- (4) $b = \text{false}$ and $B_1 \Downarrow_o \text{false}$ by (3) and def of BL1.
- (5) $B_1 \rightarrow_o^* \text{false}$ by (4) and I.H.
- (6) $B_1 \& B_2 \rightarrow_o^* \text{false} \& B_2$ by (5) and Lemma 2
- (7) $\text{false} \& B_2 \rightarrow_o \text{false}$, Bs1
- (8) $B_1 \& B_2 \rightarrow_o^* \rightarrow_o \text{false}$, by (7) and (8)

2nd Case

- (3) We applied BL2.
- (4) $B_1 \Downarrow_o \text{true}$, $B_2 \Downarrow_o b$ by (3) and def of BL2.
- (5) $B_1 \rightarrow_o^* \text{true}$ by first conjunct of (4) and I.H.
- (6) $B_2 \rightarrow_o^* b$ by second conjunct of (4) and I.H.
- (7) $B_1 \& B_2 \rightarrow_o^* \text{true} \& B_2$ by (5) and Lemma 2
- (8) $\text{true} \& B_2 \rightarrow_o B_2$, by BS1
- (9) $B_1 \& B_2 \rightarrow_o^* b$, by (7), (8), (6)

Inductive Case - 2:

Take $B \in \text{Bool}$, arbitrary.

Inductive Hypothesis: $\forall b \in \mathbb{B}. [B \Downarrow_o b \implies B \rightarrow_o^* b]$

To Show: $\forall b \in \mathbb{B}. [\neg B \Downarrow_o b \implies \neg B \rightarrow_o^* b]$

Proof of Inductive Case -2:

Take $b \in \mathbb{B}$, arbitrary.

- (1) Assume that $\neg B \Downarrow_o b$.
- (2) We obtained (1) through application of BL3 or BL4, (by inversion on $\Downarrow_o$).
- ... rest is similar to the Inductive Case - 1, and using lemma 2.

□

We will now see that we need an induction principle which is not based on the structure of the term being executed, but on the structure of the execution (or derivation).

As a motivation, assume that we added further rules to the $\rightarrow_o$ -rules and $\Downarrow_o$ -rules from earlier. And assume that these rules do not have the property that the premise is in some sense “smaller” than the conclusion.

When we attempt to expand our earlier from of $(*)$, we notice that we need to consider a further case to the inductive case, namely the case where we had applied rule BL6. Now, we cannot make progress!

What we will need is to find something that is in some sense “smaller” than the current configuration....

Induction over Derivations – Motivation

Consider an “enhanced” small-step semantics, $\rightarrow_e$, which consists of the rules in slide 22, and in addition, the rule BS7:

$$\text{BS7} \frac{}{B_1 \& B_2 \rightarrow_e B_2 \& B_1}$$

Slide 26

Similarly, consider an “enhanced” big-step semantics, $\Downarrow_e$, which consists of the rules in slide 23, and in addition, the rule BL6:

$$\text{BL6} \frac{B_2 \& B_1 \Downarrow_e b}{B_1 \& B_2 \Downarrow_e b}$$

Does

(**) $\forall B \in \text{Bool}. \forall b \in \mathbb{B}. [B \Downarrow_e b \implies B \rightarrow_e^* b]$
hold?

More details about $\rightarrow_e$

Essentially, we defined a new relation, $\rightarrow_e \subseteq \text{Bool} \times \mathbb{B}$:

$$\text{BS1} \frac{}{\text{false} \& B_2 \rightarrow_e \text{false}}$$

$$\text{BS2} \frac{}{\text{true} \& B_2 \rightarrow_e B_2}$$

$$\text{BS3} \frac{B_1 \rightarrow_e B'_1}{B_1 \& B_2 \rightarrow_e B'_1 \& B_2}$$

$$\text{BS4} \frac{B \rightarrow_e B'}{\neg B \rightarrow_e \neg B'}$$

$$\text{BS5} \frac{}{\neg \text{true} \rightarrow_e \text{false}}$$

$$\text{BS6} \frac{}{\neg \text{false} \rightarrow_e \text{true}}$$

$$\text{BS7} \frac{}{B_1 \& B_2 \rightarrow_e B_2 \& B_1}$$

Slide 27

More details about $\Downarrow_e$

Similarly, we defined a new relation $\Downarrow_e \subseteq Bool \times \mathbb{B}$

$$\text{BL1} \frac{B_1 \Downarrow_e \text{false}}{B_1 \& B_2 \Downarrow_e \text{false}}$$

$$\text{BL2} \frac{B_1 \Downarrow_e \text{true} \quad B_2 \Downarrow_o b}{B_1 \& B_2 \Downarrow_e b}$$

$$\text{BL3} \frac{B \Downarrow_e \text{true}}{\neg B \Downarrow_e \text{false}}$$

$$\text{BL4} \frac{B \Downarrow_e \text{false}}{\neg B \Downarrow_e \text{true}}$$

$$\text{BL5} \frac{}{b \Downarrow_e b}$$

$$\text{BL6} \frac{B_2 \& B_1 \Downarrow_e b}{B_1 \& B_2 \Downarrow_e b}$$

Slide 28

Proving (**) by induction over the structure of B

Proof by induction over the structure of B .

Inductive Case - 1: Take $B_1, B_2 \in Bool$, arbitrary.

Inductive Hypothesis $\forall b_1 \in \mathbb{B}. [B_1 \Downarrow_e b_1 \implies B_1 \rightarrow^* b_1] \wedge$
 $\forall b_2 \in \mathbb{B}. [B_2 \Downarrow_e b_2 \implies B_2 \rightarrow^* b_2]$

To Show: $\forall b \in \mathbb{B}. [B_1 \& B_2 \Downarrow_e b \implies B_1 \& B_2 \rightarrow^* b]$

Proof of Inductive Case-1:

Take $b \in \mathbb{B}$, arbitrary.

(1) Assume that $B_1 \& B_2 \Downarrow_e b$.

(2) We obtained (1) through application of BL1 or BL2, **or BL6** (by inversion on $\Downarrow_e$).

1st Case (3) We applied BL1.

(4) $\therefore$

2nd Case (3) We applied BL2.

(4) $\dots$

3rd Case (3) We applied BL6.

(4) $B_2 \& B_1 \Downarrow_e b$ by case

(5) ???

Slide 29

To address these problems, we apply the same techniques we learnt in the Reasoning course, where we encountered a similar situation. The solution was the application of a new induction principle, based on the derivation of a relation. We will apply similar ideas here.

Deja vu?

Motivation: The "mystery" function M - 4

```
M = M'(m, 0, 1)
M' (i, i, acc) = acc
M' (i, cnt, acc) = M'(i, cnt+1, 2*acc)
```

We want to establish

Assrt. 2: $\forall m, cnt, acc, p : \mathbb{N}. [M'(m, cnt, acc) = p \rightarrow p = 2^{(m-cnt)} * acc]$

Slide 30

Slide 31

Motivation: The "mystery" function M - 3

```

M = M'(m, 0, 1)
M' (i, i, acc) = acc
M' (i, cnt, acc) = M'(i, cnt+1, 2*acc)

```

We want to establish

Assrt. 2: $\forall m, cnt, acc, p : \mathbb{N}. [M'(m, cnt, acc) = p \rightarrow p = 2^{(m-cnt)} * acc]$

Challenge: M' defined in terms of *larger*, rather than *smaller* values.

Approaches

- **1st Approach** Find some *measure* which decreases with each recursive call.
- **2nd Approach** Explicitly count the number of recursive calls in the execution of M'.
- **3rd Approach** New induction principle.

Deja vu? – revisiting M as a big/small-steps semantics

$\Downarrow_M \subseteq (\mathbb{N} \times \mathbb{N} \times \mathbb{N}) \times \mathbb{N}$

ML1 $\frac{}{(i, i, acc) \Downarrow_M acc}$ ML2 $\frac{(i, cnt + 1, 2 * acc) \Downarrow_M p}{(i, cnt, acc) \Downarrow_e p} \quad i \neq cnt$

$\rightarrow_M \subseteq (\mathbb{N} \times \mathbb{N} \times \mathbb{N}) \times ((\mathbb{N} \times \mathbb{N} \times \mathbb{N}) \cup \mathbb{N})$

MS1 $\frac{}{(i, i, acc) \rightarrow_M acc}$ MS2 $\frac{}{(i, cnt, acc) \rightarrow_M (i, cnt + 1, 2 * acc)} \quad i \neq cnt$

We want to prove:

- $\forall i, cnt, acc, m \in \mathbb{N}. [(i, cnt, acc) \Downarrow_M m \implies m = 2^{i-cnt} * acc]$
- $\forall i, cnt, acc, m \in \mathbb{N}. [(i, cnt, acc) \rightarrow_M^* m \implies m = 2^{i-cnt} * acc]$

Slide 32

Deja vu? – inductive principle for $\Downarrow_M$

Assume a relation $Q \subseteq \mathbb{N} \times \mathbb{N} \times \mathbb{N} \times \mathbb{N}$:

$$\forall i, acc \in \mathbb{N}. [Q(i, i, acc, acc)]$$

$\wedge$

$$\forall i, cnt, acc, p \in \mathbb{N}. [Q(i, cnt + 1, 2 * acc, p) \implies Q(i, cnt, acc, p)]$$

$\implies$

$$\forall i, cnt, acc, p \in \mathbb{N}. [(i, cnt, acc) \Downarrow_M p \implies Q(i, cnt, acc, p)]$$

Slide 33

As an exercise, we apply the inductive principle for $\Downarrow_M$, to prove that

$$(Asrt1) \quad \forall i, cnt, acc, p \in \mathbb{N}. [(i, cnt, acc) \Downarrow_M p \implies p = 2^{i-cnt} * acc]$$

Proof. By induction over the structure of $\Downarrow_M$.

Base Case:

$$\text{To Show: } \forall i, acc, p \in \mathbb{N}. [acc = 2^{i-i} * acc]$$

Proof of Base Case

...

Inductive Case :

Take $i, cnt, acc, p \in \mathbb{N}$, arbitrary.

(1) $i \neq cnt$

$$\text{Inductive Hypothesis: } p = 2^{i-(cnt+1)} * 2 * acc$$

$$\text{To Show: } p = 2^{i-cnt} * acc$$

Proof of Inductive Case:

....

□

Inductive Principle over $\Downarrow_e$

$$\begin{aligned}
 & \forall B_1, B_2 \in \text{Bool}. [Q(B_1, \text{false}) \implies Q(B_1 \& B_2, \text{false})] \\
 & \wedge \\
 & \forall B_1, B_2 \in \text{Bool}. \forall b \in \mathbb{B}. [Q(B_1, \text{true}) \wedge Q(B_2, b) \implies Q(B_1 \& B_2, b)] \\
 & \wedge \\
 & \forall B \in \text{Bool}. [Q(B, \text{true}) \implies Q(\neg B, \text{false})] \\
 & \wedge \\
 & \forall B \in \text{Bool}. [Q(B, \text{false}) \implies Q(\neg B, \text{true})] \\
 & \wedge \\
 & \forall b \in \mathbb{B}. [Q(b, b)] \\
 & \wedge \\
 & \forall B_1, B_2 \in \text{Bool}. \forall b \in \mathbb{B}. [Q(B_2 \& B_1, b) \implies Q(B_1 \& B_2, b)] \\
 & \implies \\
 & \forall B \in \text{Bool}. \forall b \in \mathbb{B}. [B \Downarrow_e b \implies Q(B, b)]
 \end{aligned}$$

Slide 34

1.2 Proving (**)

We now prove the assertion,

$$(**) \quad \forall B \in \text{Bool}. \forall b \in \mathbb{B}. [B \Downarrow_e b \implies B \rightarrow_e^* b],$$

But this time we will do the proof by induction over the definition of $\Downarrow_e$. This allows us to complete the proof; something we were unable to do when applying induction on the definition of B .

Proof. By induction over the structure of $\Downarrow_e$.

If we match assertion (*) with the inductive principle for $\Downarrow_e$, we have that $Q(B, b) \equiv B \rightarrow_e^* b$.

Inductive Case - 1:

Take $B_1, B_2 \in \text{Bool}$, arbitrary.

Inductive Hypothesis: $B_1 \rightarrow_e^* \text{false}$

To Show: $B_1 \& B_2 \rightarrow_e^* \text{false}$

Proof of Inductive Case-1:

- (1) $B_1 \& B_2 \rightarrow_e^* \text{false} \& B_2$ by I.H. and Lemma 2
- (2) $\text{false} \& B_2 \rightarrow_e \text{false}$, by Bs1
- (3) $B_1 \& B_2 \rightarrow_e^* \rightarrow_e \text{false}$, by (1) and (2)

Inductive Case - 2:

Take $B_1, B_2 \in \text{Bool}, b \in \mathbb{B}$ arbitrary.

Inductive Hypothesis: $B_1 \rightarrow_e^* \text{true} \wedge B_1 \rightarrow_e^* b$

To Show: $B_1 \& B_2 \rightarrow_e^* b$

Proof of Inductive Case-2:

- (1) $B_1 \& B_2 \rightarrow_e^* \text{true} \& B_2$ by I.H. and Lemma 2
- (2) $\text{true} \& B_2 \rightarrow_e B_2$, by Bs1
- (3) $B_1 \& B_2 \rightarrow_e^* \rightarrow_e b$, by (1), (2) and I.H.

Inductive Case - 3:

Take $B \in Bool$ arbitrary.

Inductive Hypothesis: $B \rightarrow_e^* true$

To Show: $\neg B \rightarrow_e^* false$

Proof of Inductive Case-4:

- (1) $\neg true \rightarrow_e false$ by Bs5
- (3) $\neg B \rightarrow_e^* false$, by (1), and I.H.

Inductive Case - 4:

analogous to Inductive Case-3. omitted

Base Case

To Show: $\forall b \in \mathbb{B}. b \rightarrow_e^* b$

Proof of Base Case:

follows from the reflexive nature of $\rightarrow_e^*$.

Inductive Hypothesis: $B \rightarrow_e^* true$

To Show: $\neg B \rightarrow_e^* false$

Inductive Case - 5:

Take $B_1, B_2 \in Bool, b \in \mathbb{B}$ arbitrary.

Inductive Hypothesis: $B_2 \& B_1 \rightarrow_e^* b$

To Show: $B_1 \& B_2 \rightarrow_e^* b$

Proof of Inductive Case-5:

- (1) $B_1 \& B_2 \rightarrow_e B_2 \& B_1$ by Bs6
- (3) $B_1 \& B_2 \rightarrow_e^* b$, by (1), and I.H.

□

Proving (**) by induction on $\Downarrow_e$

Inductive Case - 1:

Take $B_1, B_2 \in Bool$, arbitrary.

Inductive Hypothesis: $B_1 \rightarrow_e^* false$

To Show: $B_1 \& B_2 \rightarrow_e^* false$

Proof of Inductive Case-1:

- (1) $B_1 \& B_2 \rightarrow_e^* false \& B_2$ by I.H. and Lemma 2
- (2) $false \& B_2 \rightarrow_e false$, by Bs1
- (3) $B_1 \& B_2 \rightarrow_e^* false$, by (1) and (2)

Inductive Case - 2:

...

Inductive Case - 5:

Take $B_1, B_2 \in Bool, b \in \mathbb{B}$ arbitrary.

Inductive Hypothesis: $B_2 \& B_1 \rightarrow_o^* b$

To Show: $B_1 \& B_2 \rightarrow_o^* b$

Proof of Inductive Case-5:

- (1) $B_1 \& B_2 \rightarrow_o B_2 \& B_1$ by Bs6
- (3) $B_1 \& B_2 \rightarrow_o^* b$, by (1), and I.H.

Comparing the induction principles

Slide 36

(**) could *not* be proven by induction on B , and could be proven by induction on $\Downarrow_o$

(*) *can* be proven by induction on B , and *can also* be proven by induction on $\Downarrow_o$.

We will compare the two proofs of (*) on the next slide.

Slide 37

by induct on struct. of B

Inductive Case - 1:

Take $B_1, B_2 \in \mathbb{Bool}$, arbitrary.

Inductive Hypothesis:

$$\forall b_1 \in \mathbb{B}. [B_1 \Downarrow_o b_1 \implies B_1 \rightarrow_o^* b_1] \wedge \\ \forall b_2 \in \mathbb{B}. [B_2 \Downarrow_o b_2 \implies B_2 \rightarrow_o^* b_2]$$

Show: $\forall b \in \mathbb{B}. [B_1 \& B_2 \Downarrow_o b \implies B_1 \& B_2 \rightarrow_o^* b]$

Proof of Inductive Case-1:

Take $b \in \mathbb{B}$, arbitrary.

- (1) Assume that $B_1 \& B_2 \Downarrow_o b$.
- (2) We obtained (1) by applying BL1 or BL2, (by invrs. $\Downarrow_o$).

1st Case

- (3) We applied BL1.
- (4) $b = \text{false}$, and $B_1 \Downarrow_o \text{false}$ by (3) and def of BL1.
- (5) $B_1 \rightarrow_o^* \text{false}$ by (4) and I.H.
- (6) $B_1 \& B_2 \rightarrow_o^* \text{false} \& B_2$ by (5) and Lma 2
- (7) $\text{false} \& B_2 \rightarrow_o \text{false}$, by BS1
- (8) $B_1 \& B_2 \rightarrow_o^* \rightarrow_o \text{false}$, by (7) and (8)

2nd Case

- (3) We applied BL2.
- (4) $B_1 \Downarrow_o \text{true}$, $B_2 \Downarrow_o b$ by (3) and BL2.
- (5) $B_1 \rightarrow_o^* \text{true}$ by first conjunct of (4) and I.H.
- (6) $B_2 \rightarrow_o^* b$ by second conjunct of (4) and I.H.
- (7) $B_1 \& B_2 \rightarrow_o^* \text{true} \& B_2$ by (5), Lma 2
- (8) $\text{true} \& B_2 \rightarrow_o B_2$, by BS1
- (9) $B_1 \& B_2 \rightarrow_o^* \rightarrow_o b$, by (7), (8), (6)

by induct. on struct of $\Downarrow_o$

Inductive Case - 1:

Take $B_1, B_2 \in \mathbb{Bool}$, arbitrary.

Inductive Hypothesis: $B_1 \rightarrow_o^* \text{false}$

Show: $B_1 \& B_2 \rightarrow_o^* \text{false}$

Proof of Inductive Case-1:

- (1) $B_1 \& B_2 \rightarrow_o^* \text{false} \& B_2$ by I.H., Lma 2
- (2) $\text{false} \& B_2 \rightarrow_o \text{false}$, by BS1
- (3) $B_1 \& B_2 \rightarrow_o^* \text{false}$, by (1) and (2)

Inductive Case - 2:

Take $B_1, B_2 \in \mathbb{Bool}, b \in \mathbb{B}$ arbitrary.

Induct. Hypthesis: $B_1 \rightarrow_o^* \text{true} \wedge B_2 \rightarrow_o^* b$

Show: $B_1 \& B_2 \rightarrow_o^* b$

Proof of Inductive Case-2:

- (1) $B_1 \& B_2 \rightarrow_o^* \text{true} \& B_2$ by I.H., Lma 2
- (2) $\text{true} \& B_2 \rightarrow_o B_2$, by BS1
- (3) $B_1 \& B_2 \rightarrow_o^* b$, by (1), (2) and I.H.

Transitive Closures – Many Steps of Evaluation**Properties of many steps of execution**

How can we prove, eg that

$$\forall E, E' [E \rightarrow^* E' \implies Q(E, E')]$$

where $Q \subseteq SimpleExp \times SimpleExp$.

We could apply induction on the number of steps involved in $\rightarrow^*$, or more elegantly, apply induction on the definition of $\rightarrow^*$.

Slide 38

What is the transitive closure?

Given a set A , and a relation $R \subseteq A \times A$, we define $R^+ \subseteq A \times A$, and $R^* \subseteq A \times A$ as

- $R^*(a, a') \triangleq a = a' \vee \exists a'' \in A. [R(a, a'') \wedge R^*(a'', a')]$
- $R^+(a, a') \triangleq R(a, a') \vee \exists a'' \in A. [R(a, a'') \wedge R^+(a'', a')]$

R^* and R^+ are inductively defined, and therefore give rise to inductive principles.

Slide 39

Transitive closures – defined through rules

The definition from the previous slide is equivalent with that in the current slide:

$$\begin{array}{ll}
 \text{R}^*\text{-A} \frac{}{R^*(a, a)} a \in A & \text{R}^*\text{-B} \frac{R(a, a'') \quad R^*(a'', a')}{R^*(a, a')} \\
 \text{R}^+\text{-A} \frac{R(a, a')}{R^+(a, a')} & \text{R}^+\text{-B} \frac{R(a, a'') \quad R^+(a'', a')}{R^+(a, a')}
 \end{array}$$

Slide 40

Inductive Principle for R^* and R^+

Assume a set A , and any relations $R, Q \subseteq A \times A$.

$$\begin{aligned}
 & \forall a \in A. Q(a, a) \quad \wedge \\
 & \forall a, a', a'' \in A. [R(a, a'') \wedge Q(a'', a') \implies Q(a, a')] \\
 & \implies \\
 & \forall a, a' \in A. [R^*(a, a') \implies Q(a, a')]
 \end{aligned}$$

$$\begin{aligned}
 & \forall a, a' \in A. [R(a, a') \implies Q(a, a')] \quad \wedge \\
 & \forall a, a', a'' \in A. [R(a, a'') \wedge Q(a'', a') \implies Q(a, a')] \\
 & \implies \\
 & \forall a, a' \in A. [R^+(a, a') \implies Q(a, a')]
 \end{aligned}$$

Slide 41

Note that the only difference between the inductive principle for R^* , and the inductive principle for R^+ is the base case. This is so, because R^* is reflexive, and therefore

$\forall a, a' \in A. [R^*(a, a') \implies Q(a, a')]$ implies that $\forall a \in A. Q(a, a)$, while R^+ is not reflexive, and therefore

$\forall a, a' \in A. [R^+(a, a') \implies Q(a, a')]$ does not imply that $\forall a \in A. Q(a, a)$,

Question: Would the following be a correct inductive principle?

$$\begin{aligned}
 & \forall a \in A. Q(a, a) \quad \wedge \\
 & \forall a, a', a'' \in A. [Q(a', a'') \wedge R(a'', a') \implies Q(a, a')] \\
 & \implies \\
 & \forall a, a' \in A. [R^*(a, a') \implies Q(a, a')]
 \end{aligned}$$

Slide 42

Lemma

$$\forall E_1, E'_1, E_2 \in \text{SimpleExp.}$$

$$[E_1 \rightarrow^* E'_1 \implies (E_1 +_s E_2) \rightarrow^* (E'_1 +_s E_2)]$$

The Lemma can be rewritten as

$$\forall E_1, E'_1 \in \text{SimpleExp.}$$

$$[E_1 \rightarrow^* E'_1 \implies \forall E_2 \in \text{SimpleExp.} [(E_1 +_s E_2) \rightarrow^* (E'_1 +_s E_2)]]$$

and the above has the form

$$\forall E_1, E'_1 \in \text{SimpleExp.}$$

$$[E_1 \rightarrow^* E'_1 \implies Q(E_1, E_2)]$$

where we define Q as $Q(E, E') \triangleq \forall E_2 \in \text{SimpleExp.} [(E +_s E_2) \rightarrow^* (E' +_s E_2)]$

We can therefore apply the inductive principle from slide 41

Proof of Lemma
Lemma

$$\forall E_1, E'_1 \in \text{SimpleExp}. [E_1 \rightarrow^* E'_1 \implies \\ \forall E_2 \in \text{SimpleExp}. [(E_1 +_s E_2) \rightarrow^* (E'_1 +_s E_2)]]$$

Slide 43
Proof by induction on the def. of trans closure of $\rightarrow^*$.

Base Case:
To Show: $\forall E \in \text{SimpleExp}.$

$$[\forall E_2 \in \text{SimpleExp}. [(E +_s E_2) \rightarrow^* (E +_s E_2)]]$$

Proof of Base Case

 follows from reflexivity of $\rightarrow^*$
Inductive Case:

 Take $E_1, E'_1, E''_1 \in \text{SimpleExp}$, arbitrary.

$$(1) \quad E_1 \rightarrow E''_1$$

Inductive Hypothesis: $\forall E_2 \in \text{SimpleExp}. [E''_1 +_s E_2 \rightarrow^* E'_1 +_s E_2]$
To Show: $\forall E_2 \in \text{SimpleExp}. [E_1 +_s E_2 \rightarrow^* E'_1 +_s E_2]$
Proof of Inductive Case:

 Take $E_2 \in \text{SimpleExp}$, arbitrary

$$(2) \quad E''_1 +_s E_2 \rightarrow^* E'_1 +_s E_2, \quad \text{by I.H.}$$

$$(3) \quad E_1 +_s E_2 \rightarrow E''_1 +_s E_2, \quad \text{from (1) and def. of } \rightarrow \text{ for SimpleExp.}$$

$$(4) \quad E_1 +_s E_2 \rightarrow^* E'_1 +_s E_2, \quad \text{from (2) and (3).}$$

Slide 44

Slide 45**Summary: Proofs by Induction over Relations**

- State over which entity is the induction – this requires thought.
- Base Case (s): State what is to be shown
- Prove the Base Case (s)
 - State what is taken arbitrary – if any
 - Justify each step
- Inductive Step (s)
 - State what is taken arbitrary (at least one thing is).
 - State the Inductive Hypothesis.
 - State what is to be shown.
- Prove the Inductive Step (s).
 - state what is taken arbitrary – if any.
 - Justify each step.
- Not all lemmas require a proof by induction.

Slide 46**Summary: Proofs**

The summary for induction over terms is the same as that for induction over relations.

In exams and in the coursework you will be expected to satisfy the points listed in the summaries.